ProjectPulse AI

AI-Assisted Project Operations and Health Monitoring System

Assignment: *Associate Project Management Intern – NxtWave*
Submitted by: *Samanuri Sri Manasa Varma*

1. Problem Statement

Project-related information is often scattered across spreadsheets, chat conversations, and individual updates. Managers spend considerable time collecting progress reports, identifying blockers, following up with employees, and preparing stakeholder summaries.

This creates three major problems:

- Limited visibility into the actual status of projects.
- Delayed identification of risks, blockers, and dependencies.
- Manual effort in consolidating information and deciding priorities.

I wanted to address this by building a centralized system that helps managers understand project health, identify tasks requiring attention, and make informed decisions with AI assistance.

2. Proposed Solution – ProjectPulse AI

ProjectPulse AI is a full-stack project operations command center that combines task management, explainable risk scoring, AI recommendations, and a contextual Project Copilot.

The objective is to reduce manual monitoring effort and help managers focus on important project decisions.

Key Capabilities

1. **Centralized Dashboard**

Provides an overview of tasks, progress, priorities, and project-level metrics in one place.

2. **Task Management**

Allows users to create and edit tasks, assign owners, update progress, set deadlines, and maintain task history.

3. **Explainable Risk Intelligence**

Evaluates task status, due dates, progress, and dependency-related signals to calculate risk scores. Tasks are classified as Low, Medium, High, or Critical.

Each risk classification is supported by identifiable reasons.

4. **AI Recommendation Workflow**

Generates suggested actions for tasks requiring attention. Recommendations enter an approval queue where managers can approve or reject them.

5. **AI Project Copilot**

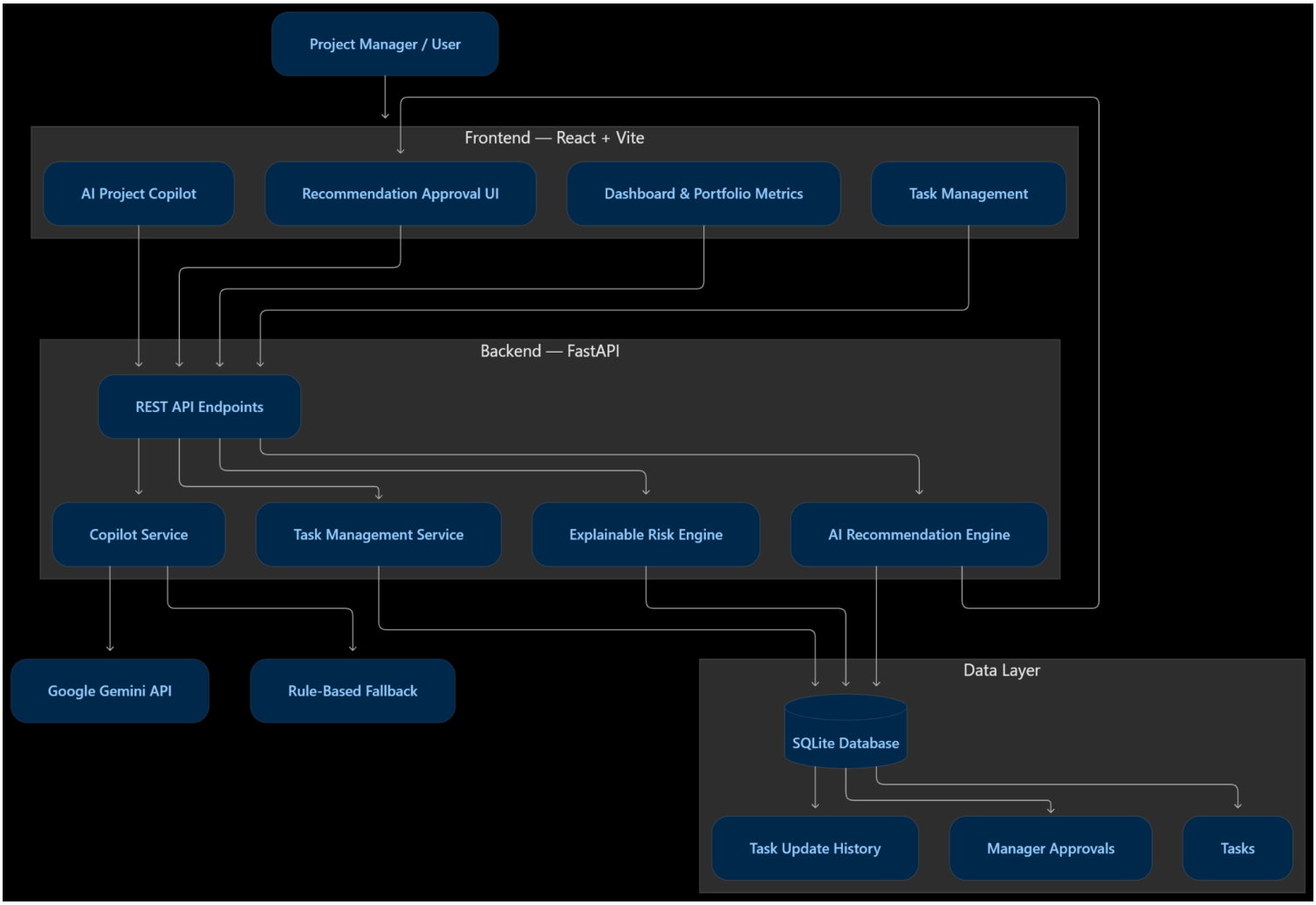
Allows managers to ask questions such as:

- Which tasks need immediate attention?
- What are the major project risks?
- Which tasks are blocked?
- What actions should be prioritized?

The Copilot uses Google Gemini when configured and deterministic rule-based fallback logic when Gemini is unavailable.

Important: The risk engine is heuristic decision support, not a trained predictive ML model.

3. Workflow Diagram and Architecture



Technology Stack

Component	Technology
Frontend	React, Vite, JavaScript
Styling	CSS
Backend	Python, FastAPI
Database	SQLite, SQLAlchemy
AI Integration	Google Gemini API
API Documentation	Swagger UI

Architecture Explanation

- Frontend: Handles dashboard visualization, task operations, Copilot interaction, and approvals.
- Backend: Processes API requests, task updates, risk calculations, and recommendations.
- Database: Stores tasks, update history, and manager approval decisions.
- AI Layer: Uses Gemini for contextual responses, with a rule-based fallback.
- Risk Engine: Calculates explainable risk scores using task conditions.

The current prototype uses manually entered task information and fictional sample data. External integrations such as Slack, Teams, and Jira are not implemented.

4. Reflection

Why did I choose this solution?

I chose project health monitoring because managers need more than task completion updates. They need to understand which tasks are delayed, why they are delayed, who is responsible, and what action should happen next.

I wanted to build something that supports these decisions rather than creating another basic task tracker.

Assumptions Made

1. Employees can submit task updates through a centralized interface.

2. Task owners, deadlines, priorities, and progress are available as structured information.
3. Task conditions can help identify potential risks.
4. Managers should review important AI recommendations before action.
5. Gemini integration should be optional, with fallback functionality.

What Would I Improve With Another Week?

- Automated updates: Explore Slack or Teams integration to reduce manual reporting.
- Context validation: Validate task information before presenting AI-generated blockers as confirmed facts.
- Advanced risk intelligence: Incorporate historical task performance and structured dependencies.
- Reporting: Generate standardized weekly project health summaries.
- User feedback: Allow managers to correct inaccurate recommendations.

5. Stage 2 – 30-Day Business Review

After one month of piloting the solution across three teams, leadership identified challenges related to adoption, AI accuracy, reporting consistency, and metrics.

5.1 Top Five Challenges

Challenge	Why It May Be Happening
Low adoption (65%)	Employees may consider updates an additional task rather than part of their existing workflow.
AI context inaccuracies	Missing or outdated information can lead to incorrect interpretations.
Inconsistent reporting	Teams use different formats and definitions for project updates.
Workflow bypassing	Employees may find direct chat updates quicker and more convenient.
Unreliable metrics	Incomplete and inconsistent updates affect reporting accuracy.

5.2 Top Three Priorities

I would prioritize the following:

1. **Employee Adoption**

Without consistent usage, the system cannot provide reliable project visibility.

2. **AI Accuracy**

Incorrect AI recommendations can reduce manager trust and lead to poor decisions.

3. **Reporting Standardization**

Consistent input data is essential for meaningful metrics and accurate project analysis.

These three challenges affect the effectiveness of the entire workflow and should be addressed before scaling.

5.3 Redesigned Solution

I would improve ProjectPulse through five changes:

1. **Standardized Employee Updates**

Introduce a simple update format:

- What was completed?
- What is currently in progress?
- Are there any blockers?
- Is an external dependency pending?
- What is the expected completion date?

2. **AI Context Validation**

Before identifying a blocker, the system should check whether sufficient supporting information exists. If information is missing, it should request clarification instead of making assumptions.

3. **Human Approval**

Managers should review important recommendations before they are accepted or acted upon.

4. **Common Reporting Standards**

Introduce consistent definitions for On Track, In Progress, At Risk, Blocked, and Completed across all teams.

5. **Weekly Project Reports**

Generate summaries covering task completion, overdue tasks, high-risk tasks, unresolved blockers, and pending approvals.

5.4 Five Success Metrics

Metric	Measurement
Workflow Adoption Rate	Percentage of employees consistently submitting updates.
Update Completeness Rate	Percentage of updates containing all required information.
AI Recommendation Acceptance Rate	Percentage of recommendations approved by managers.
Risk Identification Accuracy	Percentage of identified risks confirmed by managers.
On-Time Task Completion Rate	Percentage of tasks completed by their deadlines.

I would establish baseline values during the pilot and compare them against results after implementing improvements.

6. AI Design Log

6.1 AI Tools Used

- ChatGPT: Problem analysis, workflow design, development assistance, debugging, API testing, and documentation.
- Google Gemini API: Contextual responses through the Project Copilot.

6.2 Initial Workflow Prompt

"Design an AI-powered project management workflow that helps managers monitor task progress, identify blockers, understand project risks, and generate recommendations while reducing manual reporting effort."

6.3 Refined Workflow Prompt

"Build a full-stack AI-assisted project operations system combining centralized task management, explainable risk scoring, AI recommendations, and a contextual Project Copilot. Maintain task history, store manager approval decisions, and include human review before recommendations are acted upon. Provide deterministic fallback logic when the external AI service is unavailable."

6.4 Development Iterations

Approximately 10 major development iterations were involved, covering:

1. Problem identification and workflow planning.
2. Frontend dashboard development.
3. FastAPI backend implementation.

4. SQLite database integration.
5. Task creation, editing, and history.
6. Explainable risk engine development.
7. Gemini API integration.
8. Recommendation and approval workflow.
9. API validation and functional testing.
10. Documentation and final refinements.

6.5 Knowledge Sources

- NxtWave assignment brief.
- React and FastAPI documentation.
- SQLAlchemy documentation.
- Google Gemini API documentation.
- Application task data, database records, and API responses.

The prototype uses fictional sample project information.

6.6 Validation and Learning

I performed manual functional and API testing.

Test Scenario	Result
Task creation and editing	Passed
Risk assessment	Passed
AI Copilot responses	Passed
Recommendation generation	Passed
Approval and rejection	Passed

Task history	Passed
Database persistence	Passed
Invalid progress validation (422)	Passed
Missing task handling (404)	Passed
Frontend refresh	Passed

One AI suggestion I did not adopt:

I did not allow AI to automatically execute all recommendations or escalate tasks. Incorrect interpretations could create unnecessary escalations. I retained manager approval for important decisions.

One significant improvement:

Adding explainable risk scoring and the recommendation approval workflow transformed the initial task-tracking concept into a decision-support system.

Conclusion

Building ProjectPulse AI helped me understand that making a workflow AI-native is not simply about adding an AI chatbot.

The real value comes from connecting information, identifying meaningful patterns, supporting decisions, and keeping humans involved where judgment matters.

My objective was to build a practical prototype that helps managers spend less time collecting updates and more time making informed decisions.